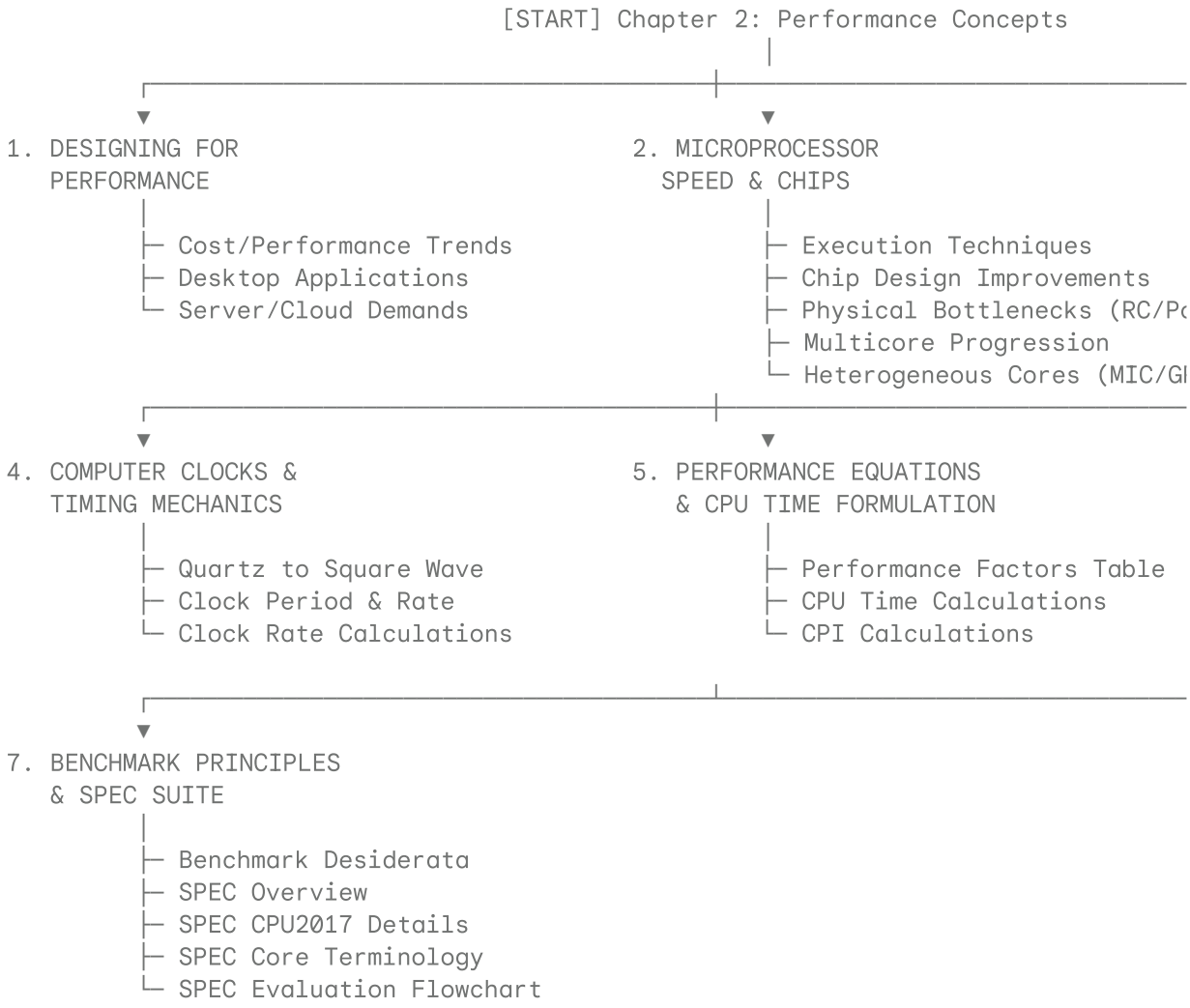
MIND MAP: Chapter 2 - Performance Concepts

William Stallings: Computer Organization and Architecture (11th Edition)



1. Designing for Performance (Context & Trends)

- **Cost vs. Performance Dynamics**
  - System cost continues to drop dramatically.
  - Performance and capacity continue to rise equally dramatically.
  - *Benchmark Comparison*: Modern laptops possess the raw computing power of an IBM mainframe from 10 to 15 years ago.
  - *Commoditisation*: Microprocessors have become so inexpensive that some are designed as disposable devices ("microprocessors we throw away").
- **Power-Demanding Desktop Applications**

    - Image processing
    - Three-dimensional (3D) rendering
    - Speech recognition
    - Videoconferencing
    - Multimedia authoring
    - Voice and video annotation of files
    - Simulation modeling
- **Enterprise & Infrastructure Demands**
    - *Business Servers*: Increasingly powerful systems needed to handle database and transaction processing, supporting massive client/server networks that replace legacy centralized mainframe centers.
    - *Cloud Service Providers (CSPs)*: Use massive, high-performance server banks to satisfy high-volume, high-transaction-rate applications for a broad, diverse spectrum of clients.

2. Microprocessor Speed & Chip Organization

- **Contemporary Processor Speed-Up Techniques**
    - **Pipelining**: Operates like an assembly line; processor moves data or instructions into a conceptual pipe with all stages of the pipe processing simultaneously.
    - **Branch Prediction**: Processor looks ahead in the fetched instruction code from memory to predict which branches (or groups of instructions) are likely to be processed next.
    - **Superscalar Execution**: The ability to issue multiple instructions in every single processor clock cycle (effectively utilizing multiple parallel pipelines).
    - **Data Flow Analysis**: Processor analyzes instruction lists to identify which instructions depend on each other's results or data, creating an optimized execution schedule.
    - **Speculative Execution**: Uses branch prediction and data flow analysis to execute instructions before they actually appear in the normal step-by-step program flow. Results are stored in temporary locations to keep execution engines operating at maximum capacity.
- **Improvements in Chip Organization and Architecture**
    - **Increasing Hardware Speed**: Fundamentally achieved by shrinking logic gate size.
        - Allows more gates to be packed tightly together.
        - Increases maximum clock rate.
        - Reduces electrical propagation time for signals across wires.
    - **Increasing Cache Size and Speed**: Dedicating a significant portion of the processor chip area directly to cache memory, which reduces cache access times significantly.
    - **Altering Processor Organization**: Shifting designs to increase the effective speed of instruction execution (primarily through parallelism).
- **Problems with Clock Speed and Logic Density**
    - **Power Wall**: Power density increases exponentially with logic density and higher clock speeds. This causes extreme difficulties in dissipating generated heat.
    - **RC Delay**: The speed of electron flow is limited by the resistance (R) and capacitance (C) of the metal wires interconnecting components.
        - As sizes shrink, wire interconnects become thinner, increasing resistance (R).

        - As wires are packed closer together, capacitance (C) increases.
        - Delay increases proportionally as the product $R \times C$ increases.
    - **Memory Wall**: Memory access speed (latency) and transfer speed (throughput) lag far behind processor speed improvements.
- **Processor Trends (1970–2010 Transition)**
    - *Transistors*: Sustained exponential growth (doubling regularly).
    - *Frequency (MHz) & Power (W)*: Experienced steep growth until the mid-2000s, where they hit a hard limit (flattening out due to the power wall).
    - *Cores*: Transitioned from single-core architectures to multi-core architectures around 2005 to sustain performance growth within power budgets.
- **Multicore Architectures**
    - *Concept*: Integrating multiple processors on the same chip to increase performance without needing to raise the clock rate.
    - *Strategy*: Utilize multiple simpler processors on a single chip rather than one highly complex, power-hungry processor.
    - *Cache Allocation*: Having multiple processors on-chip justifies larger cache designs. As on-chip caches grew, architects introduced structured Hierarchical Caches (progressing to two, and now three levels of cache on a single chip).
- **Heterogeneous Computing Cores**
    - **Many Integrated Core (MIC)**:
        - Integrates a very large number of homogeneous, general-purpose cores on a single silicon chip.
        - Offers a massive leap in raw performance.
        - Presents major software engineering challenges in designing software that can exploit such vast thread parallelism.
    - **Graphics Processing Unit (GPU)**:
        - Engineered specifically to perform highly parallel operations on graphics data.
        - Historically located on discrete plug-in graphics cards to encode, render 2D/3D graphics, and process video.
        - Modern usage: Used as vector processors for general computation applications that require highly repetitive, mathematical calculations.

3. Performance Balance

- **Definition**: Adjusting system organization and architecture to compensate for the operational mismatch among the performance capabilities of different physical components (primarily the processor vs. memory/peripheral gap).
- **Architectural Mitigation Strategies**
    1. **Wider DRAM Architectures**: Increase the number of bits retrieved during a single access by designing DRAM chips to be "wider" rather than "deeper," supported by wider system bus data paths.
    2. **Complex Cache Hierarchies**: Reduce the overall frequency of access to main memory by integrating increasingly complex and efficient cache structures between the processor and DRAM.

3. **Buffered/Cached DRAM Interfaces**: Redesign the DRAM interface to include high-speed cache or buffer logic directly on the DRAM chip itself.
4. **High-Speed Bus Interconnects**: Raise the interconnect bandwidth between processors and memory through high-speed buses and structured hierarchies of buses that buffer and organize data flows.

- **Typical I/O Device Data Rates (Figure 2.1)**
  - *Keyboard*: $\approx 10^2$ bps
  - *Mouse*: $\approx 3 \times 10^2$ bps
  - *Scanner*: $\approx 10^7$ bps
  - *Laser printer*: $\approx 1.5 \times 10^7$ bps
  - *Optical disc*: $\approx 5 \times 10^8$ bps
  - *Hard disk*: $\approx 10^9$ bps
  - *Wi-Fi modem (Max speed)*: $\approx 10^{10}$ bps
  - *Graphics display*: $\approx 1.2 \times 10^{10}$ bps
  - *Ethernet modem (Max speed)*: $\approx 10^{11}$ bps

4. Computer Clocks & Timing Mechanics

- **System Clock Generation (Figure 2.5)**

$$\text{Quartz Crystal} \xrightarrow{\text{Analog Sine Wave}} \text{A-to-D Converter} \xrightarrow{\text{Digital Square Wave}} \text{System Clock I}$$

- **Definitions & Formulas**
  - **System Clock**: Runs at a constant rate; determines when hardware execution events occur.
  - **Clock Period (τ) / Cycle Time**: The exact amount of physical time that elapses for one complete clock period (e.g., 5 ns).
  - **Clock Rate (f)**: The inverse of the clock cycle time (τ):

$$f = \frac{1}{\tau}$$

  - *Calculation Example*: If a computer has a clock cycle time (τ) of 5 ns, its clock rate (f) is:

$$f = \frac{1}{5 \times 10^{-9} \text{ sec}} = 200 \times 10^6 \text{ Hz} = 200 \text{ MHz}$$

5. Quantitative Performance Formulation & CPU Time

- **Performance Factors and System Attributes (Table 2.1)**
  - *Key Variables*:
    - I_c = Instruction Count
    - p = Cycles per instruction (or processor implementation variables)
    - m = Memory access clock cycles

- k = Ratio of memory accesses to instruction references
- τ = Clock cycle time

- *Matrix of Influence*:

Attribute / Component	Instruction Count (I_c)	Cycles per Instruction (p)	Memory Cycles (m)	Access Ratio (k)	Clock Period (τ)
Instruction Set Architecture (ISA)	X	X			
Compiler Technology	X	X	X		
Processor Implementation		X			X
Cache & Memory Hierarchy				X	X

- **Computing CPU Time**
  - *Basic Formulation*:

$$\text{CPU Time} = \text{CPU Clock Cycles} \times \tau$$

  - *Using Clock Rate (f)*:

$$\text{CPU Time} = \frac{\text{CPU Clock Cycles}}{f}$$

  - *Deriving Clock Cycles*:

$$\text{CPU Clock Cycles} = I_c \times \text{CPI}$$

  - *Complete CPU Time Equation*:

$$\text{CPU Time} = I_c \times \text{CPI} \times \tau = \frac{I_c \times \text{CPI}}{f}$$

  - *Dimensional Unit Analysis Verification*:

$$\text{seconds} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{clock cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{clock cycle}}$$

- **CPU Time Calculation Examples**
  - *Example 1 (Calculating Absolute Time)*:
    - Given: $f = 50$ MHz, $I_c = 1,000$ instructions, $\text{CPI} = 3.5$.
    - Calculation:

$$\text{CPU Time} = \frac{1000 \times 3.5}{50 \times 10^6 \text{ Hz}} = 7 \times 10^{-5} \text{ seconds} = 70 \ \mu\text{s}$$

- *Example 2 (Calculating Speedup Factor)*:
  - Given: f increases from 200 MHz to 250 MHz (all other variables remain identical).
  - Calculation:

$$\text{Speedup} = \frac{\text{CPU Time}_{\text{old}}}{\text{CPU Time}_{\text{new}}} = \frac{f_{\text{new}}}{f_{\text{old}}} = \frac{250 \text{ MHz}}{200 \text{ MHz}} = 1.25 \times \text{ faster}$$

- *Simplifying Assumptions*: These direct models assume idealized execution, meaning they ignore memory latency gaps, cache misses, and pipeline stalls.

- **Computing Cycles Per Instruction (CPI)**
  - CPI represents the statistical average number of cycles required to execute a single instruction.
  - If instruction types have known frequencies (F_i) and dedicated cycles (CPI_i):

$$\text{CPI} = \sum_{i=1}^{n} (\text{CPI}_i \times F_i)$$

  - *Standard Distribution Example*:

Instruction Class (Op)	Frequency (F_i)	Cycle Cost (CPI_i)	Weighted Contribution ($\text{CPI}_i \times F_i$)	Percentage of Total Execution Time
ALU	50% (0.50)	1	0.5	23%
Load	20% (0.20)	5	1.0	45%
Store	10% (0.10)	3	0.3	14%
Branch	20% (0.20)	2	0.4	18%
Total	100%		2.2	100%

6. Performance Metrics & Marketing Pitfalls

- **MIPS (Millions of Instructions Per Second)**
  - *Formula*:

$$\text{MIPS} = \frac{I_c}{\text{Execution Time} \times 10^6}$$

  - *Example*: A program executing 3 million instructions in 2 seconds has a MIPS rating of:

$$\text{MIPS} = \frac{3 \times 10^6}{2 \times 10^6} = 1.5 \text{ MIPS}$$

  - *Advantage*: Very simple to understand and measure.
  - *Disadvantage*: Fails to reflect real-world performance because simpler instruction sets execute more instructions per second but complete less actual work per instruction.
- **MFLOPS (Millions of Floating-Point Operations Per Second)**
  - *Formula*:

$$\text{MFLOPS} = \frac{\text{Floating-Point Operations}}{\text{Execution Time} \times 10^6}$$

  - *Example*: A program completing 4 million FP operations in 5 seconds has an MFLOPS rating of:

$$\text{MFLOPS} = \frac{4 \times 10^6}{5 \times 10^6} = 0.8 \text{ MFLOPS}$$

  - *Advantage*: Simple to understand and measure for mathematical computation limits.
  - *Disadvantage*: Shares the same flaws as MIPS and is restricted only to floating-point code blocks.
- **The "Clock Frequency is Performance" Fallacy**
  - The general public often equates raw clock frequency (f) directly with performance, ignoring instruction count (I_c) and cycle efficiency (CPI).
  - *Which processor is faster?*
    - **Processor A**: $\text{CPI} = 2$, $f = 5 \text{ GHz}$ ($\text{Execution factor} \propto \frac{2}{5} = 0.40$)
    - **Processor B**: $\text{CPI} = 1$, $f = 3 \text{ GHz}$ ($\text{Execution factor} \propto \frac{1}{3} = 0.33$)
    - *Result*: Processor B is actually **faster** (assuming identical ISA and compilers), despite having a lower clock speed.
  - *Historical Proofs*:
    - An 800 MHz Intel Pentium III outperformed a 1 GHz Intel Pentium 4.
    - Intel Core i7 is faster clock-for-clock than the older Intel Core 2.

7. Benchmark Principles & SPEC Assessment

- **Desirable Characteristics of Benchmark Programs**
  1. Written in a high-level language, making them highly portable across different target architectures.
  2. Representative of a specific programming domain or paradigm (e.g., systems programming, numerical modeling, commercial transactions).
  3. Easily and reliably measured.
  4. Have wide, open-source distribution.
- **System Performance Evaluation Corporation (SPEC)**

- **Benchmark Suite**: A collected group of high-level language programs that provide a representative, standard test of a computer's performance in a given application area.
- **SPEC Consortium**: An industry group that defines and maintains standardized benchmark suites widely used for commercial system comparison and research.

- **SPEC CPU2017 Suite**
  - The industry standard suite for evaluation of processor-intensive applications.
  - Designed for computation-bound workloads (applications spending the vast majority of execution time on CPU math rather than system I/O).
  - Consists of 20 **integer benchmarks** and 23 **floating-point benchmarks** written in C, C++, and Fortran.
  - Includes separate **rate** and **speed** options for all integer and most floating-point tests.
  - Contains over 11 **million lines of code**.
  - *Note on Kloc*: Refers to source line count (including comments/whitespace) divided by $1,000$.
- **SPEC CPU2017 Benchmark Classifications (Table 2.5)**
  - **Integer Benchmarks (Examples)**:
    - *500/600.perlbench*: Perl Interpreter (Language: C, Kloc: 363)
    - *502/602.gcc*: GNU C Compiler (Language: C, Kloc: 1304)
    - *505/605.mcf*: Route Planning (Language: C, Kloc: 3)
    - *520/620.omnetpp*: Discrete Event Network Simulation (Language: C++, Kloc: 134)
    - *523/623.xalancbmk*: XML to HTML via XSLT (Language: C++, Kloc: 520)
    - *525/625.x264*: Video Compression (Language: C, Kloc: 96)
    - *531/631.deepsjeng*: AI Chess Alpha-Beta Search (Language: C++, Kloc: 10)
    - *541/641.leela*: AI Go Monte Carlo Search (Language: C++, Kloc: 21)
    - *548/648.exchange2*: AI Sudoku Solution Generator (Language: Fortran, Kloc: 1)
    - *557/657.xz*: General Data Compression (Language: C, Kloc: 33)
  - **Floating-Point Benchmarks (Examples)**:
    - *503/603.bwaves*: Explosion Modeling (Language: Fortran, Kloc: 1)
    - *507/607.cactuBSSN*: Physics; General Relativity (Language: C/C++/Fortran, Kloc: 257)
    - *508.namd*: Molecular Dynamics (Language: C/C++, Kloc: 8)
    - *510.parest*: Biomedical Imaging; Optical Tomography (Language: C++, Kloc: 427)
    - *511.povray*: Ray Tracing (Language: C++, Kloc: 170)
    - *519/619.lbm*: Fluid Dynamics (Language: C, Kloc: 1)
    - *521/621.wrf*: Weather Forecasting (Language: Fortran/C, Kloc: 991)
    - *526.blender*: 3D Rendering & Animation (Language: C++, Kloc: 1577)
    - *527/627.cam4*: Atmosphere Modeling (Language: Fortran/C, Kloc: 407)
    - *628.pop2*: Wide-scale Climate Ocean Modeling (Language: Fortran/C, Kloc: 338)
    - *538/638.imagick*: Image Manipulation (Language: C, Kloc: 259)
    - *544/644.nab*: Molecular Dynamics (Language: C, Kloc: 24)

    - *549/649.fotonik3d*: Computational Electromagnetics (Language: Fortran, Kloc: 14)
    - *554/654.roms*: Regional Ocean Modeling (Language: Fortran, Kloc: 210)
- **SPEC Terminology**
    - **Benchmark**: A standardized high-level language program compiled and executed on any system implementing the required compiler.
    - **System Under Test (SUT)**: The specific hardware/software computer system being evaluated.
    - **Reference Machine**: A baseline computer used by SPEC to establish reference baseline times (T_{ref}) for each benchmark.
    - **Base Metric**: Required for official publication; compiles code using very strict, standard guidelines to ensure a level playing field.
    - **Peak Metric**: Optional; allows developers to apply compiler optimizations to maximize hardware performance.
    - **Speed Metric**: Measures the raw time required to execute a single compiled benchmark task. Used to compare single-task processing capability.
    - **Rate Metric**: Measures throughput or capacity (how many simultaneous tasks are completed over a given period). This metric allows the SUT to execute concurrent tasks to make full use of multiple processors or cores.
- **SPEC Evaluation Workflow (Figure 2.7)**
    1. **Start** the evaluation process.
    2. **Get next program** in the benchmark suite.
    3. **Run program three times** on the System Under Test (SUT).
    4. **Select the median execution value** from the three runs (T_{SUT}).
    5. **Calculate the performance ratio** for the program:

$$\text{Ratio}(\text{prog}) = \frac{T_{\text{ref}}(\text{prog})}{T_{\text{SUT}}(\text{prog})}$$

    6. **Are there more programs left in the suite?**
        - *Yes*: Loop back to get the next program.
        - *No*: **Compute the Geometric Mean** of all calculated program ratios.
    7. **End** of evaluation process (resulting in a final composite SPEC index).

8. Theoretical Performance Laws & Statistical Means

(Consolidating key concepts highlighted in the Chapter Summary)

- **System Performance Laws**
    - **Amdahl's Law**: Mathematically models the potential speedup of a system when only a fraction of it is improved. Shows that the speedup is limited by the serial (unimprovable) portion of the task:

$$\text{Speedup} = \frac{1}{(1 - f_{\text{enhanced}}) + \frac{f_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

- **Little's Law**: A fundamental queuing theory equation used to analyze system behavior in a steady state. It relates the average number of items in a system (L) to the average arrival rate (λ) and the average time (W) an item spends in the system:

$$L = \lambda \times W$$

- **Statistical Means for Computer Systems**
  - **Arithmetic Mean**: Standard average, useful for tracking progress when tasks represent equal quantities of work, but can be highly distorted by outliers:

$$\text{AM} = \frac{1}{n} \sum_{i=1}^{n} X_i$$

  - **Harmonic Mean**: Primarily used to average rates (such as MIPS or execution rates) to prevent mathematical skewing:

$$\text{HM} = \frac{n}{\sum_{i=1}^{n} \frac{1}{X_i}}$$

  - **Geometric Mean**: Used by SPEC to normalize benchmark results. It treats all benchmarks equally, regardless of the size of their absolute running times, preventing any single benchmark from dominating the final score: